D_I: where the resources go

Analysis and selectable hls4ml experiments · 10 October 2026

D_I's BRAM is dominated by full-feature-map FIFOs. Its DSPs are dominated by the resource-strategy convolutions, especially conv5 and conv6. The strongest first experiments are FIFO depth profiling and higher reuse in the late layers. Both preserve the trained model in intent; exact output parity still needs validation.

New baseline: R_S preprocessing and standard hls4ml CNN, with the D_I direct interface and timestamps. The model is pinned; firmware is regenerated per iteration. D_I's custom conv0 kernel is excluded. The measurements below describe historical D_I, not the new baseline.

Measured routed scope	BRAM tiles	DSPs	URAM
D_I user wrapper	354 (17.56%)	1,328 (14.72%)	11
CNN IP within wrapper	328	1,312	11
CNN inter-layer FIFOs	250.5	0	11
CNN convolution arithmetic	77.5	1,300	0

Selected independent experiments

Option	Change	Why it is worth testing
A · first BRAM test	Vitis FIFO depth optimization	Targets 70.8% of wrapper BRAM without changing weights.
B2 · first DSP target	conv5/conv6: Resource, RF144	First-order wrapper estimate: 880 DSPs (9.75%); modest late-stage work, but tail grows.
A+B3 · later	Re-profile with broader late reuse	Deferred; not included in the current build campaign.
B1 / B3 · alternatives	RF96 late-only / broader late reuse	Trade less tail growth against more DSP reduction.

Status: A and B2 selected. Independent iteration tooling is implemented and baseline regeneration/validation is underway. A/B2 routed builds follow baseline correctness; no board programming or benchmarks are part of this campaign.

The 10% BRAM/DSP caps are inherited from the existing campaign, not a newly imposed requirement. Current routed timing passes at 250 MHz, with whole-design WNS +0.044 ns. Preserve timing as a separate acceptance gate. [E1–E4]

1 · Baseline, scope and provenance

Analyzed image: D_I, recovery/route_01/bitstreams/cyt_top.bit. The hardware selection and the 520-call parity evidence identify the same image. The routed report is Vivado 2024.2, Physopt postRoute, 9 October 2026. Report and key generated-source SHA-256 hashes were checked against the existing manifests. [E1–E3]

Image SHA-256: 3fc293ce1ee9c8027e86b877c0449b01d77ca8c357ccbe3ed1beafce92cde3b7

Scope	LUT	FF	BRAM tiles	DSP
D_I full device	187832	258354	496.0	1328
D_I user wrapper	67143	86435	354.0	1328
D_N full device	124080	174381	162.0	0
D_N user wrapper	7788	12616	20.0	0
Full-device D_I − D_N	63752	83973	334.0	1328
Wrapper D_I − D_N	59355	73819	334.0	1328

• BRAM tile = RAMB36 + RAMB18/2. D_I wrapper: 348 RAMB36 + 12 RAMB18 = 354 tiles. HLS reports BRAM_18K instead; do not compare their raw counts directly.

• Device totals: 2,016 BRAM tiles; 9,024 DSPs; 960 URAM; 1,303,680 LUTs; 2,607,360 FFs. Wrapper percentages use whole-device capacity.

• The D_N delta includes changed shell/interface placement and implementation. Wrapper attribution is a hierarchy measurement; it is not the same quantity as full-design incremental overhead.

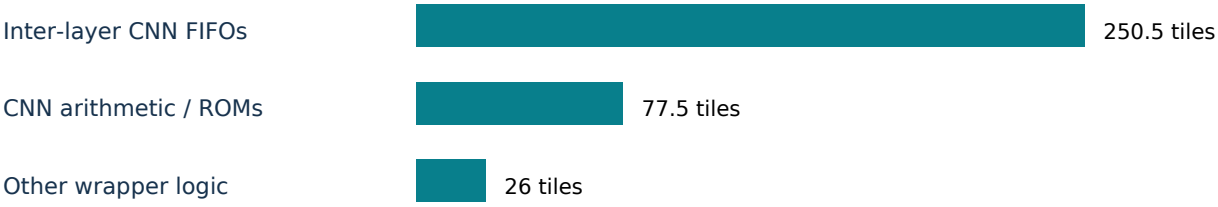
Existing caps, expressed as counts

At ≤10% of the device: ≤201.5 realizable BRAM tiles and ≤902 DSPs. Relative to D_I, this requires at least **152.5 fewer BRAM tiles** and **426 fewer DSPs**. If non-CNN wrapper cost stays fixed at 26 tiles and 16 DSPs, the CNN budgets are ≤175.5 tiles and ≤886 DSPs.

Do not use C-synthesis estimates as the final resource count

The existing CNN HLS report estimates 233 BRAM_18K (=116.5 tile equivalents), 652 DSPs and 0 URAM. Routed CNN attribution is 328 tiles, 1,312 DSPs and 11 URAM. For example conv1 is estimated at 0 DSPs but routes to 231. Later mapping changes memory and arithmetic implementation substantially; this analysis therefore uses routed hierarchy for all baseline attribution. [E5]

2 · BRAM: deep intermediate FIFOs



The 32-token FIFO between preprocessing and CNN uses **zero BRAM** (10 LUTRAMs). The large allocations are internal to the CNN: generated stream depths equal entire feature-map sizes. Streaming preprocessing removed the old external frame buffer but retained these internal queues. [E2, E6]

Stream / producer	Depth × logical bits	BRAM tiles	URAM
layer2 / pad_conv0	67,600 × 16	48	0
layer3 / conv0	16,384 × 240	105	0
layer5 / act0	16,384 × 64	28.5	0
layer6 / pool0	4,096 × 64	0	1
layer7 / pad_conv1	4,356 × 64	10.5	0
layer8 / conv1	4,096 × 384	0	6
layer10 / act1	4,096 × 128	0	2
layer11 / pool1	1,024 × 128	4	0
layer12 / pad_conv2	1,156 × 128	0	2
layer13 / conv2	1,024 × 600	17	0
layer15 / act2	1,024 × 192	5.5	0
layer16,17 / pool2,pad3	256,324 × 192	6 combined	0
layer18 / conv3	256 × 600	8.5	0
layer20,22 / act3,pad4	256,100 × 192	6 combined	0
layer23 / conv4	64 × 800	11.5	0

Remaining inter-layer FIFOs consume no BRAM/URAM; their LUTRAM, SRL and register costs remain relevant. Logical widths precede synthesis bit trimming and physical memory packing. Conv0's 30-bit × 8-channel pre-activation output is much wider than its following 8-bit activation. "W8A8" does not mean every stream is eight bits wide.

Opportunity: the first three BRAM FIFOs alone use 181.5 tiles. All CNN inter-layer FIFOs total 250.5 tiles (76.4% of CNN BRAM). Meeting the wrapper cap through FIFO savings alone needs a 60.9% reduction of this allocation. Removing all FIFO BRAM would leave 103.5 tiles; that is an accounting bound, not a feasible prediction.

3 · DSPs: late convolutions and reuse

D_I retains the seven-layer 256×256 W8A8 P50 model. Conv0 has a custom window-generator/arithmetic DATAFLOW split; conv1–conv6 retain generated hls4ml kernels. Effective settings come from parameters.h, not just YAML. [E6–E8]

Layer	Output positions	Nin × Nout	Strategy / RF	DSP	BRAM
conv0	16,384	25 × 8	Latency / 1	61	0
conv1	4,096	72 × 16	Latency / 1	231	0
conv2	1,024	144 × 24	Resource / 36	96	8.5
conv3	256	216 × 24	Resource / 36	144	12
conv4	64	216 × 32	Resource / 36	192	15.5
conv5	16	288 × 32	Resource / 32	288	21
conv6	4	288 × 32	Resource / 32	288	20.5
output_dense	1	32 × 1	Latency / 1	12	0
Preprocessing	65,536 pixels	Index arithmetic	Outside CNN	16	0

conv5 + conv6 = 576 DSPs (43.4% of wrapper DSPs). conv2–conv6 together use 1,008 DSPs (75.9%). Early conv0 is only 61 DSPs; slowing it risks undoing the latency optimization for little DSP benefit.

Why 50% pruning does not halve the late-layer DSP count

For the current Resource kernel at RF ≤ Nin, the unrolled multiplier-bank size is Nin × Nout / RF. Conv5: 288 × 32 / 32 = 288. This agrees exactly with its routed DSP count. The kernel iterates over dense weights and reshapes the ROM; it does not use the sparsity-adjusted multiplier_limit field from parameters.h to halve this bank. Individual zeros in a reused bank are not equivalent to an entirely removable multiplier. [E7]

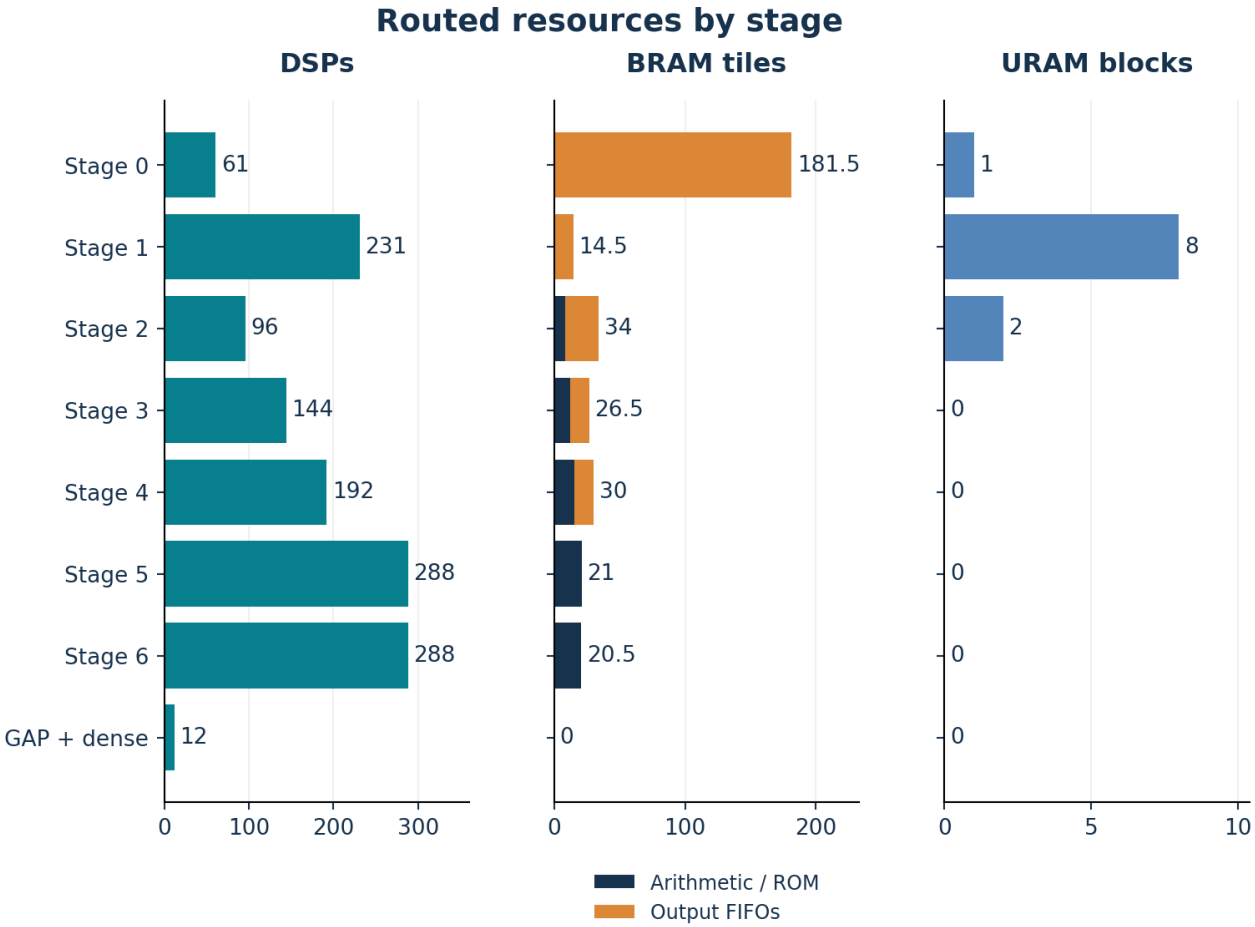
conv2–conv6's 77.5 BRAM tiles sit inside their dense-resource arithmetic hierarchy. Weight arrays are reshaped and bound to ROM_nP_BRAM. Parallel reads create wide, shallow storage: conv2's HLS ROM entry alone is width 766 × depth 36. Increasing reuse can reduce ROM banking as well as DSPs, but physical BRAM savings are quantized and must be measured.

Requested RF32 is not always effective RF32

The manualA YAML requests RF32 for conv2–conv6. The backend legalizes conv2–conv4 to RF36. Candidate factors in this report were checked against the installed backend's validity predicate. Always record the generated effective factors; blanket RF64 would also be legalized for several layers.

3a · Resource overview by CNN stage

Each stage groups pad_convN, convN, actN and poolN. Output FIFOs are charged to their producing layer. These disjoint allocations reconcile to **328 CNN BRAM tiles, 1,312 DSPs and 11 URAM**. Remaining wrapper cost: 26 BRAM tiles and 16 DSPs. [E2, E6]



Stage = padding + convolution + activation + pooling, including producer FIFOs.

Where the optimization opportunity is largest

• **BRAM: stage 0 owns 181.5 tiles (55.3% of CNN BRAM), all in its inter-layer FIFOs.** Optimize FIFO depth first; reducing conv0's arithmetic is not the way to recover this memory.

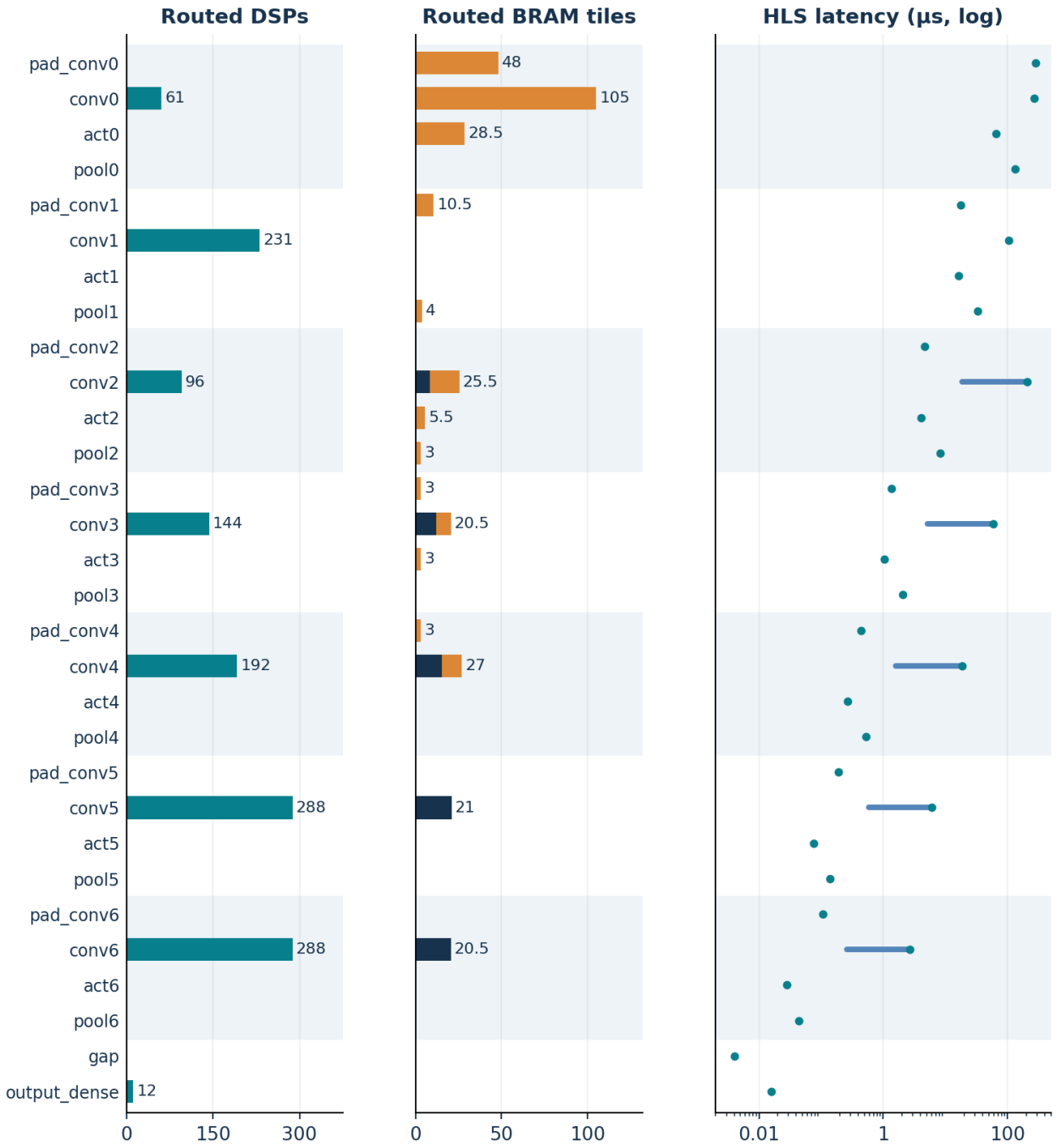
• **DSP: conv5/conv6 own 576 DSPs (43.9% of CNN DSPs).** They have little frame-level work compared with early layers, making them the first reuse-factor candidates.

• **URAM: stages 0–2 own all 11 blocks.** A BRAM-only view would miss these sizeable activation queues. Re-profile them alongside the BRAM-backed FIFOs.

The BRAM plot separates arithmetic/weight-ROM allocation from output-FIFO allocation. Stage totals describe physical ownership, not a claim that every stored bit is necessary. Standalone plots are saved as PDF and PNG alongside the report.

3b · Every layer: area and latency together

Resource bars are routed counts; latency is an HLS estimate at 4 ns. Lines show the reported minimum–maximum per module; dots mark the maximum. The logarithmic latency axis keeps small layers visible. These are not confidence intervals or measured board latencies. [E2, E5]



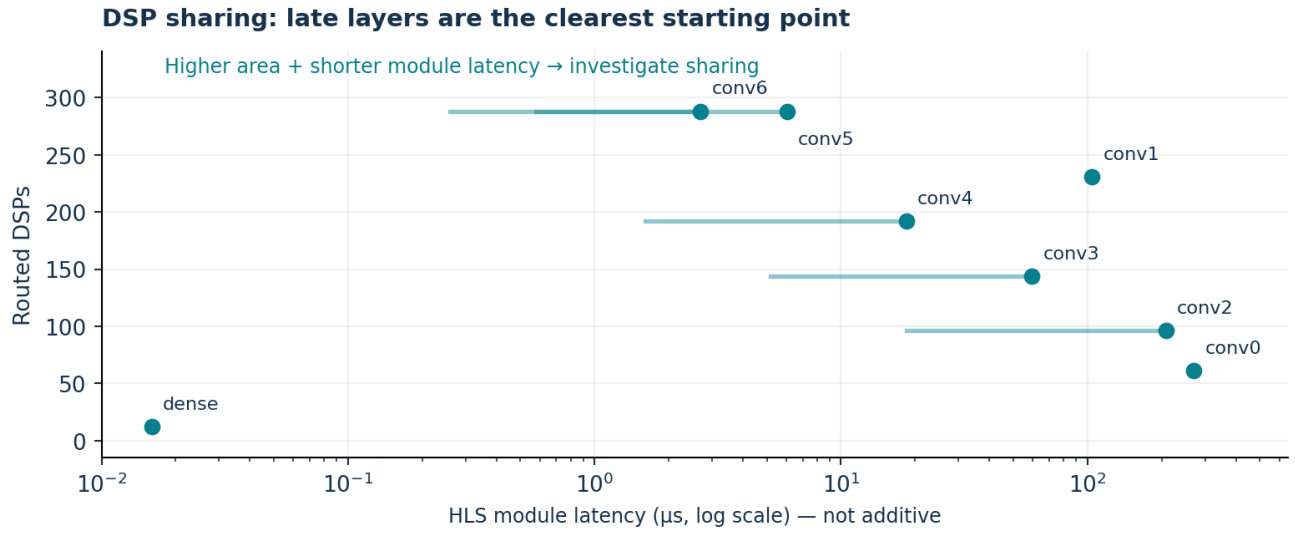
BRAM: dark = arithmetic / ROM; orange = producer output FIFO.

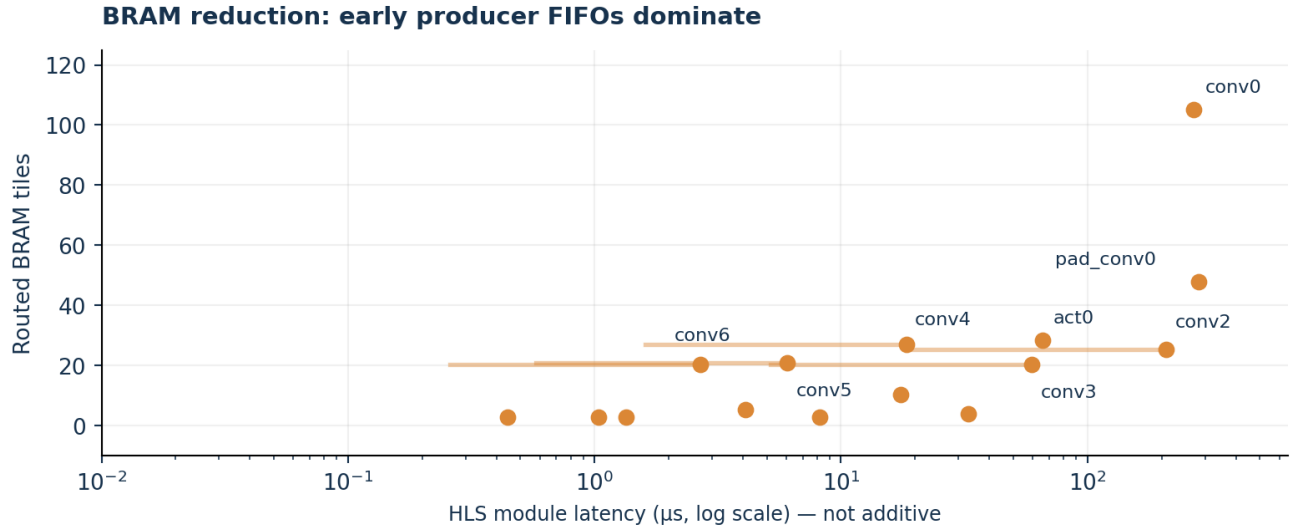
Latency: line = HLS min–max; dot = maximum. Concurrent modules: do not sum.

Do not sum the latency column or interpret it as percentages of end-to-end latency. DATAFLOW layers execute concurrently. The board captures CNN boundary events, not per-layer execution. Marginal latency cost of a resource change needs a new RTL/board run. FIFO cost is attributed to its producer; a deep FIFO does not itself imply a full-buffer delay.

3c · Finding the best tradeoff visually

For DSP sharing, look for **high resource use and short module latency**. For BRAM, inspect the producer's FIFO allocation as well as arithmetic. Horizontal segments are HLS reported ranges; points use the upper estimate. The axes mix measured area and estimated timing deliberately, as an opportunity screen—not a prediction of savings.





Target	Routed allocation	HLS module latency	Interpretation
conv5 / conv6	288 DSPs each	0.580–6.052 / 0.260–2.692 µs	Best first DSP-sharing targets; extra tail still matters.
conv0	61 DSPs; 105 FIFO tiles	270.432 µs	Keep fast arithmetic; resize its output FIFO.
pad_conv0 / act0	48 / 28.5 FIFO tiles	282.720 / 65.548 µs	Memory targets; latency estimates do not prove queue occupancy.
conv2	96 DSPs; 25.5 total tiles	18.500–208.084 µs	Less DSP opportunity, more throughput risk than late layers.

Long input streams can hide layer work behind transport; the final layers still affect the unhidden verdict tail. These plots support prioritizing FIFO profiling and late-layer reuse, but do not establish a safe FIFO depth or a latency-neutral RF increase.

4 · Which controls are actually available?

Control	Current project support	Assessment for D_I
Strategy + ReuseFactor	Allowed in hls.layer_tuning.layers; both fields required. Only Latency / Resource.	Primary DSP control; select individual layers.
FIFO depth optimization	hls.fifo_depth_optimization and fifo_profiling_depth; Vitis + io_stream required.	Primary BRAM control; built-in flow performs synthesis + RTL profiling.
Per-layer Precision	Native hls4ml supports it; current manual-layer schema rejects it.	New iteration configuration exposes native per-layer precision; preserve baseline numeric types and validate exact outputs.
Global accumulator precision	hls.accum_precision is exposed; applied after manual tuning.	Too coarse for first experiment; can change several layers' numerical behavior.
Output / pool precision	output_precision and pool_accum_precision exposed.	Dense and final pooling are tiny targets; low priority.
ConvImplementation: Encoded	Installed Vitis feature check disallows it and falls back to LineBuffer.	Not a supported D_I optimization knob.
Compression, unrolled strategies	Not accepted by current manual-layer schema; backend/layer restrictions apply.	Do not assume sparse Conv2D compression or DSP-free mapping is available.
ParallelizationFactor / IOType	No current per-layer exposure; model is already a stream design.	Not a first-line reduction; increasing parallelism adds resources.

Two integration details matter

• Historical D_I preparation copied archived production firmware and patched conv0. The new iteration flow regenerates standard hls4ml firmware from pinned architecture/weights and each iteration's complete configuration. Previous iteration firmware is never a build input. Historical packages remain evidence. [E8]

• New iterations exclude D_I's custom conv0 split. OPT_1_STREAMING combines R_S preprocessing and standard CNN with the D_I direct interface/timestamps. Profile with native hls4ml, then validate the final guarded wrapper. The 32-token external FIFO stays fixed.

Compatibility checked against the CLI-preferred ml_baseline/.venv_hls4ml package (1.3.0). A separate .venv_hls4ml_coyote points to an editable checkout, version 0.6.0.dev516+gd4a6a2f5. Do not silently switch environments or treat either installed version as proof of the historical generator version. Pinned production logits and synthetic boundary fixtures provide the numerical reference; generated code remains configurable. [E9, W1–W3]

5 · Option A: profile and resize CNN FIFOs

Priority: first BRAM experiment. Keep model, arithmetic precision and effective reuse factors fixed. Target internal layer2/layer3/layer5 queues first, then the remaining internal streams including the 11 URAM-backed FIFOs.

```
hls:
  backend: Vitis
  io_type: io_stream
  fifo_depth_optimization: true
  fifo_profiling_depth: 100000
```

These keys already exist in the project pipeline. The profiling depth is a deliberately large temporary bound, not a proposed deployed depth. The flow records FIFO occupancy during RTL co-simulation and emits fifo_depths.json. It requires at least two top-level invocations. The installed Vitis pass writes its parsed profile depth directly; do not assume the Vivado documentation's "maximum + 1" rule is applied here. [E9, W1]

Application to the standard R_S CNN schedule

• Regenerate an independent candidate from the pinned model using standard hls4ml kernels. Use the native Vitis FIFO pass with at least two prepared inputs, preserve its selected depths, and revalidate the final guarded wrapper.

• Exercise at least two frames, then the established reset/stall/consecutive-frame suite. Include fast pixel delivery, sparse arrival, input stalls and output backpressure. A depth sufficient for one trace is not a universal throughput guarantee.

• Keep the raw-to-CNN pixel FIFO at depth 32. It is outside the native pass and already uses no BRAM. There is no custom conv0 window FIFO in the new baseline.

• Re-profile whenever reuse factors or arithmetic scheduling change. Verify final generated depths and ensure a subsequent model.write(), copy step or build preparation does not restore full-map depths.

Expected result and decision rule

Expected: substantial BRAM and possibly URAM reduction; DSPs essentially unchanged. The targeted pool is 250.5 BRAM tiles, not a promised saving. Remaining weight-ROM cost and wrapper cost are 103.5 tiles before resized FIFO allocation. Correctness alone determines functional validity. Record area and latency without thresholds; routed timing separately determines implementation readiness. D_I's resource pool is historical context, not a new-baseline measurement.

This is not the same as hard-coding all queues to depth 2 or 32. Exact safe depths are unknown until final-schedule profiling. The observed 13/32 peak occupancy in the external pixel FIFO gives no occupancy bound for internal CNN queues. [E4]

6 · Options B1–B3: targeted DSP sharing

Keep conv0, conv1 and conv2 unchanged first. Later layers operate on progressively fewer output positions, allowing more reuse without throttling the entire front end. Their additional work still contributes to the final verdict tail.

Option	Effective RFs conv3 / conv4 / conv5 / conv6	Projected wrapper DSP	Reduction
Baseline	36 / 36 / 32 / 32	1,328 · 14.72%	—
B1 · moderate	36 / 36 / 96 / 96	944 · 10.46%	384 · 28.9%
B2 · late-only target	36 / 36 / 144 / 144	880 · 9.75%	448 · 33.7%
B3 · spread reuse	72 / 72 / 96 / 96	776 · 8.60%	552 · 41.6%

These are first-order arithmetic estimates, not HLS or routed results. Replace each changed Resource layer's measured DSP count with Nin × Nout / RF; hold all other components fixed. This model matches all five current Resource layers exactly. For B2, each late layer has 288 × 32 / 144 = 64 multiplier banks, giving 1,328 − 576 + 128 = 880 DSPs in this historical-scope estimate. Mapping, accumulation, placement and timing can change in the new build. No numerical BRAM prediction is justified from DSP scaling alone.

B2 configuration fragment

```
hls:
  layer_tuning:
    mode: manual_layers
    layers:
      conv5: {Strategy: Resource, ReuseFactor: 144}
      conv6: {Strategy: Resource, ReuseFactor: 144}
```

The new B2 iteration stores a complete configuration without inheritance; retain conv0–conv4 settings and verify resolved reuse factors. B1 uses 96 for the same two layers. B3 additionally uses Resource/RF72 for conv3 and conv4.

Latency implications

At 250 MHz, B2 adds an idealized (16 + 4) × (144 − 32) = 2,240 reuse-loop iterations, or 8.96 µs of aggregate scheduled work, across conv5/conv6. This is neither measured latency nor a bound on verdict delay: layers overlap, loops have overhead and input/output stalls matter. B3 spreads extra work to 256-position conv3 and 64-position conv4, so its tail/throughput cost may be larger.

conv2 currently needs roughly 1,024 × 36 = 36,864 reuse iterations per frame before overhead. Doubling it to RF72 gives 73,728, exceeding 65,536 raw input-pixel cycles. This is an early warning that a uniform reuse increase can create a bottleneck, not a cycle-accurate latency model. Leave conv2 unchanged in the first round.

7 · Further options and limits

Option	Concrete experiment	Benefit / cost / priority
C · conv1 Resource	conv1: Resource, RF4 or RF8; legal factors. Keep conv0 unchanged.	Potential DSP sharing, but RF4 implies 288 banks versus 231 routed DSPs today; RF8 implies 144. Can add weight BRAM and latency. Secondary.
D · narrower conv results	Extend per-layer Precision support; range/rounding study of conv0 result and accum first.	Targets the 240-bit conv0 output queue and adder widths. Profile FIFOs first; never blindly change W8A8 accumulators to 8 bits.
E · further late RF increase	conv5/conv6 RF288 only if B2 still misses area or more savings are desired.	Fewer multiplier banks, longer tail; diminishing system-level value. Retain exact numerical gates.
F · model quantization / pruning	W6/W4 or more/structured sparsity, with new training and evaluation.	Outside first-round configuration-only scope. Changes the model; exact D_I parity is no longer the acceptance objective.

Numerical precision needs a separate design argument

Generated input is ap_fixed<16,6>. Convolution accumulators/results are 30/13, 24/11, 25/12, 25/13, 25/14, 26/16 and 26/16 (total/integer bits). QReLU then narrows to unsigned 8 bits. Preserve enough fractional bits and guard bits for the existing rounding, saturation and zero-threshold decision; a smaller integer range alone is not sufficient proof of equivalence.

Use interval bounds from actual weights plus representative layer traces, then compare intermediate activations and exact final logits against D_I. Keep weights, biases and input preprocessing fixed for a hardware-only experiment. Changing precision may alter classifier output without retraining; do not label such a candidate equivalent based solely on matching class labels.

Do not prioritize these for this request

• Moving BRAM to URAM changes the memory mix; it is not a reduction in stored state. Native BRAM/URAM binding or DSP-to-LUT pragmas would be HLS/RTL mapping changes, not currently exposed hls4ml configuration controls.

• Compression=True is not a generic sparse-Conv2D switch for this Resource kernel. More unstructured pruning alone does not establish proportional DSP savings.

• Global Resource/RF changes are not monotonic improvements. Historical 256×7 sweeps include RF2 using more DSPs than RF1; those are different HLS estimates at 5 ns and cannot predict optimized D_I at 4 ns. [E10]

• The preprocessing block is only 16 DSPs and zero BRAM. Removing its cost could not meet either cap. Preserve the exact integer-linspace / padding / inversion behavior.

8 · Selection and later validation

Selected: OPT_1_STREAMING validates the R_S-based baseline through RTL; OPT_2_DSP_RESOURCES_B2 and OPT_3_BRAM_FIFO_A are independent routed builds. A+B3 is deferred. Shared tooling is allowed; model weights are shared and pinned; earlier iteration directories are not dependencies.

Gate	Required evidence after selection
Identity / isolation	Complete per-iteration config and model architecture; shared pinned weights; freshly generated firmware; source/tool/config hashes and effective RFs. No previous iteration firmware dependency.
C simulation	Exact D_I raw-preprocessing outputs and final logit bits; all established corpus and boundary cases. Preserve the first-pixel launch guard and input/output ABI.
HLS / FIFO profile	At least two invocations for occupancy; generated FIFO depths, II and latency; evaluate the final standard-kernel schedule. Report BRAM_18K separately from routed tiles.
RTL regression	Startup idle → second driver reset → frames; input/output stalls and consecutive frames. Correct pixel counts, logits, verdicts, and timestamps; no deadlock.
Implementation	Routed utilization by hierarchy, all timing checks, 250 MHz closure, LUT/FF/URAM tradeoffs as well as BRAM/DSP. Same reporting scope as D_I.
Board admission	Standard driver, exact rebuilt image, at least the existing 520-check D_I gate before benchmarking; never use reset-suppressed diagnostic driver.
Benchmark later	Same inputs and endpoints as D_I/D_N. Small/fast input tests plus the four synthetic and eight real sizes. Median/p95, calibration uncertainty, FIFO stalls and final tail.

Latency baseline and interpretation

Existing D_I host-observed medians: 0.357 ms at 4 MiB, 1.342 ms at 16 MiB, 5.281 ms at 64 MiB, and 10.534 ms near 128 MiB. Last preprocessed pixel to verdict is about 21.33 µs. The small-input smoke CNN interval is about 0.296 ms. These are different measurement scopes; neither the HLS 0.283 ms estimate nor the 21–23 µs D_I-minus-sink gap is a complete end-to-end compute time. [E4–E5]

For large inputs much computation overlaps transport. A slower CNN may preserve most of that overlap while increasing the final tail; small inputs expose throughput regressions sooner. Record latency without admission ceilings and compare scopes explicitly. D_I is a historical comparator; the new baseline removes its custom conv0 optimization. Area improvement alone is not enough to claim a better checker.

No retraining is needed in intent for options A–C. Exact numerical agreement is a testable requirement, not an assumption. If an option changes outputs, investigate before treating the historical 87.7% held-out accuracy or 88.5% F1 as applicable.

9 · Evidence index and reproduction

Paths below are repository-relative. J = hls4ml/optimize/runs/parallel_resume_20261009_1550/jobs/D_I. F = J/hw/src/hls/cnn_wrapper/firmware. R = hls4ml/optimize/reports/u55c02_20261009. Line references use the current archived text files.

E1 · Selected hardware and parity — hls4ml/optimize/runs/board_benchmark_u55c02_20261009_2345/hardware_selection.json; diagnostics/D_I_admission_01/status.json under that same run root. J/artifact_manifest.json and J/source_manifest.json. Manifest hardware_validated=false is a build-time field; later board evidence is separate.

E2 · Routed attribution — J/recovery/route_01/reports/shell_utilization_hierarchical.rpt: full device line 25; wrapper 602; CNN 908; conv0–6 914, 959, 1186, 1402, 1746, 2170, 2754; dense 3338; internal FIFOs 3352–3436; external FIFO 3942; preprocessing 3943. No parent/child double-counting in the disjoint breakdown.

E3 · Timing and sink — J/recovery/route_01/reports/shell_timing_summary.rpt: summary 140–142. D_N: hls4ml/optimize/runs/direct_stream_20261008_002352/jobs/D_N/build/reports/shell_utilization_hierarchical.rpt.

E4 · Measured performance — R/REPORT.md; R/data/D_I_event_medians.csv; R/data/all_variant_summaries.csv. MEMORY.md for reset evidence and regression requirements.

E5 · HLS estimate scope — J/build/validated_hls/cnn_wrapper/solution1/syn/report/prod_res256_manualA_coyote_accel_csynth.rpt and cnn_wrapper_csynth.xml. Per-layer csynth XML/RPT files in the same directory.

E6 · Generated graph / precisions — F/prod_res256_manualA_coyote_accel.cpp: FIFO declarations and optimized conv0 call. F/defines.h: numeric types. F/parameters.h: effective layer geometry and reuse.

E7 · Arithmetic / pruning — F/nnet_utils/nnet_dense_resource.h: dense_resource_rf_leq_nin, ARRAY_RESHAPE, ROM_nP_BRAM and ReuseLoop. Resource ROM width: dense_resource_rf_leq_nin_ap_ufixed_ap_fixed_25_12_5_3_0_config13_mult_s_csynth.rpt, line 194, in E5 directory.

E8 · Configuration / integration — hls4ml/configs/hls4ml_hand_tuning/res256_layers7_W8A8_P50_manualA.yaml; hls4ml/optimize/build_campaign/direct_prepare.py: lines 24–32; hardware/common/conv0_dataflow.hpp and frame_start.hpp in the same campaign.

E9 · Supported controls — hls4ml/pipeline/hls_layer_tuning.py (allowed keys, strategies); part3_hls.py lines 44–82 (configuration, FIFO flow); hls4ml/scripts/hls4ml_run.py (environment discovery). Installed hls4ml/backends/vitis/passes/fifo_depth_optimization.py and feature_check.py; backends/fpga/fpga_backend.py lines 253–275. Inspected under ml_baseline/.venv_hls4ml/lib/python3.10/site-packages/.

E10 · Background and prior sweeps — ML_Based_Deep_Bitstream_Inspection_for_Serverless_FPGAs_Report.pdf, chapters 3–7; README.md; hls4ml/results/selected_feasible_candidates_resource_strategy/experiment_summary.csv. Historical experiments are context, not predicted D_I results.

W1 · Official FIFO optimization documentation (consulted 2026-10-10): https://fastmachinelearning.org/hls4ml/advanced/fifo_depth.html

W2 · Official configuration documentation (consulted 2026-10-10): https://fastmachinelearning.org/hls4ml/api/configuration.html

W3 · Official implementation overview (consulted 2026-10-10): https://fastmachinelearning.org/hls4ml/ir/ir.html

Reproduce with: python3 hls4ml/optimize/reports/optimizations/build_di_resource_report.py. Requires system ReportLab, DejaVu fonts and Matplotlib in ml_baseline/.venv_hls4ml. plot_di_resources.py exports PNG/PDF figures under figures/D_I_resources/. Companion D_I_resource_evidence.json contains hierarchy rows, all 30 per-layer latency ranges and source hashes. Reads existing evidence only; no hardware builds.